Lecture 26

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

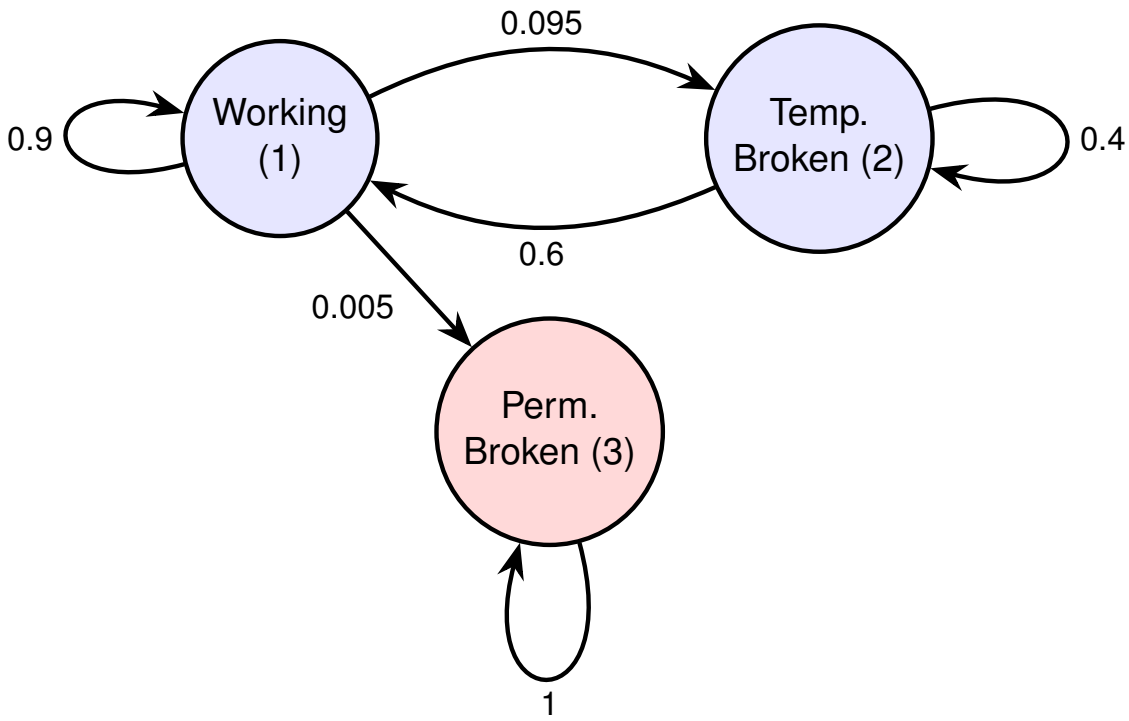
Review of Lecture 25: Markov Chains

- **Markov chain:** the next state depends only on the current state; outcomes are a discrete set of states.
- **Transition matrix** P: $P(j,i)$ = probability of going from state i to state j; each *column* sums to 1.
- **Evolution:** $v^{(n)} = P^n v^{(0)}$.
- **Broken machine** (Working / Temp. Broken / Perm. Broken): computing P^{365} gave $S(365) = 1 - (v^{(365)})_3 \approx 0.21$, i.e. about a 21% chance the machine still works after one year.
- **Steady state:** $P\mathbf{v} = \mathbf{v}$ (an eigenvector for $\lambda = 1$). Here $\mathbf{v} = [0\ 0\ 1]^T$: State 3 is an *absorbing state*.
- **Google link:** a random surfer is a Markov chain; PageRank is the limiting distribution, $x = Ax$.

Today: absorbing chains (transient states, fundamental matrix), back to PageRank, then eigenvalue methods: Power Method and SVD.

The Machine as a Graph

Recall the broken-machine chain from Lecture 25. Each arrow is labeled with its transition probability (a column of P lists the arrows leaving that state):



- Blue: *transient* states (states 1, 2). Red: the *absorbing* state (3), which has no arrow leaving it except to itself.
- The sub-matrix Q on the next slide is exactly the arrows among the blue states; R is the arrow into the red state.

Transient and Absorbing States

- **Transient states:** The machine can leave these. (Here: State 1 = Working, State 2 = Temporarily Broken)
- **Absorbing state:** Once entered, you never leave. (Here: State 3 = Permanently Broken)

Writing the transition matrix with rows as "from" states, $\tilde{P} = P^T$, we can separate it into parts:

$$\tilde{P} = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$$

where

$$Q = \begin{pmatrix} 0.9 & 0.095 \\ 0.6 & 0.4 \end{pmatrix}, \quad R = \begin{pmatrix} 0.005 \\ 0 \end{pmatrix}.$$

Q shows how the machine moves among the "non-broken" states; R gives the probability of moving into the permanently broken state.

The Fundamental Matrix

- The **fundamental matrix** is

$$N = (I - Q)^{-1} = I + Q + Q^2 + Q^3 + \cdots$$

  Each power of Q represents one more "day" of transitions before the machine is permanently broken.
- N_{ij}: expected number of days the machine is in state j if it started in state i (before it breaks forever).
- The sum of row i = average number of days until failure, starting from state i.

For our machine:

$$N = \begin{pmatrix} 200 & 31.67 \\ 200 & 33.33 \end{pmatrix}$$

- Starting from **Working**: about 232 days on average.
- Starting from **Temp. Broken**: about 233 days on average.

Will the Machine Eventually Break?

- The long-term chance of ending up in the **permanently broken** state is

$$B = NR.$$

For our machine:

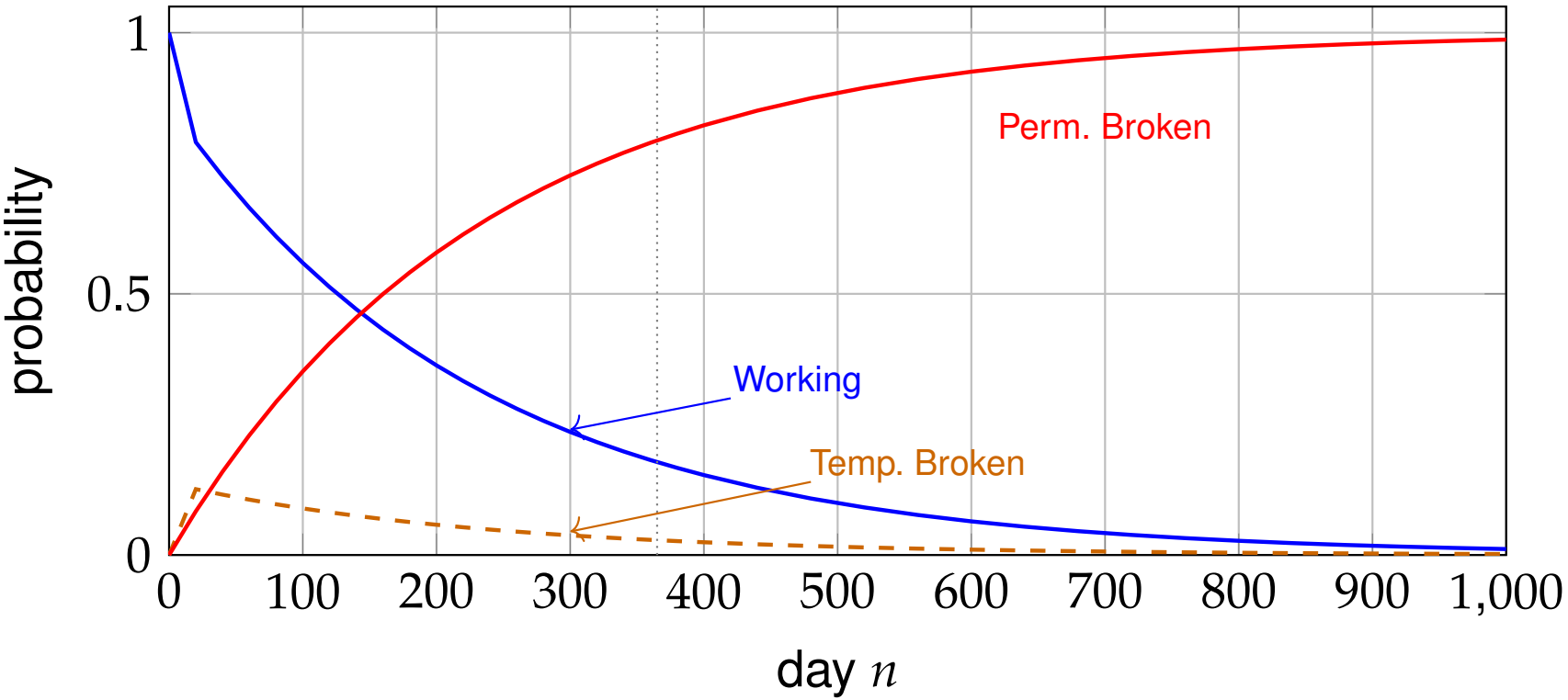
$$B = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Meaning:

- No matter where the machine starts (working or temporarily broken), it will **eventually end up permanently broken**.
- The only difference is **how long it takes**.

Visualizing $v^{(n)} = P^n v^{(0)}$

Start with $v^{(0)} = [1\ 0\ 0]^T$ (working on Day 1):



- Dotted line: day 365, where $(v^{(365)})_3 \approx 0.79$ (so $S(365) \approx 0.21$).
- The curves approach the steady state $[0\ 0\ 1]^T$: the absorbing state wins.

Try It Yourself: MATLAB or Python

MATLAB

```
P = [ .9 .6 0; .095 .4 0; .005 0 1 ];
v0 = [1; 0; 0];
v365 = P^365 * v0;        % After 365 days
survive = 1 - v365(3);    % Still functional
```

Python

```
import numpy as np
P = np.array([[.9,.6,0],[.095,.4,0],[.005,0,1]])
v0 = np.array([1,0,0.0])
v365 = np.linalg.matrix_power(P, 365) @ v0
survive = 1 - v365[2]
```

Both give about a **21% survival chance** after one year.

Back to Google

- Let W be the set of Web pages that can be reached by following a chain of hyperlinks starting from a page at Google.
- Let n be the number of pages in W.
- The set W actually varies with time, by the end of 2020, n was over 40 billion.
- Let G be the $n \times n$ connectivity matrix of W, that is, $G_{i,j}$ is 1 if there is a hyperlink from page j to page i and 0 otherwise.
- The matrix G is huge, but very sparse; its number of nonzeros is the total number of hyperlinks in the pages in W.

Google and Probability

- Let c_j and r_i be the column and row sums of G, respectively. That is,

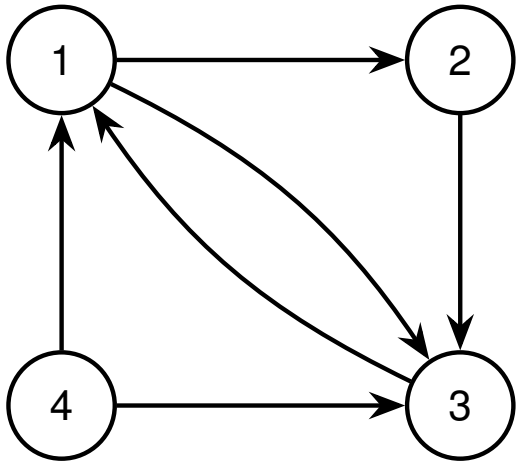
$$c_j = \sum_i G_{i,j}, \qquad r_i = \sum_j G_{i,j}$$

- Then c_k and r_k are the outdegree and indegree of the k-th page. In other words, c_k is the number of links from page k and r_k is the number of links into page k.
- Let p be the fraction of time that the random walk follows a link.
- Google typically takes this to be $p = 0.85$.
- Then $1 - p$ is the fraction of time that an arbitrary page is chosen.

Google meets Markov

- Let A be an $n \times n$ matrix whose elements are $A_{i,j} = pG_{i,j}/c_j + \delta$ where $\delta = (1-p)/n$.
- This matrix is the transition matrix of the Markov chain of a random walk!
- Notice that A comes from scaling the connectivity matrix by its column sums.
- The j-th column is the probability of jumping from the j-th page to the other pages on the Web.

A Tiny Web: PageRank by Hand



$$G = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$G_{ij} = 1$ if page j links to page i; column sums $c = (2, 1, 1, 2)$.

With $p = 0.85$, $\delta = 0.15/4 = 0.0375$, $A_{ij} = pG_{ij}/c_j + \delta$, solving $x = Ax$ (with $\sum x_i = 1$):

$$x \approx (0.380,\ 0.199,\ 0.384,\ 0.038)^T$$

Pages 1 and 3 rank highest (they link to each other and page 3 also gets links from 2 and 4). Page 4 has no in-links, so it gets only the random-jump share δ.

The problem

Can write A, the transition matrix, as

$$A = pGD + ez^T$$

where e is the vector of all ones and where ez^T accounts for dead-end pages and

$$D_{jj} = 1/c_j \text{ (or 0)} \quad z_j = \delta \text{ (or } 1/n)$$

Then $x = Ax$ can be written (scaling x so that $z^T x = 1$)

$$(I - pGD)x = (z^T x)e = e$$

see `pagerank.m`

Eigenvectors and Google

Find $x = Ax$ and the elements of x are Google's PageRank. Remember $n > 4 \times 10^{10}$ (as of 2020) and growing.

For any particular query, Google finds pages on the Web that match the query. The pages are then listed in the order of their PageRank.

Matlab

Moler's scripts

To search from a homepage, you will type a statement like:

```
[U,G] = surfer('http://usu.edu',n).
```

This starts at the given URL and tries to surf the Web until it has visited n pages. That is, an n by n matrix is formed.

Note, `surfer.m` is very primitive...

Matlab

Moler's scripts

- Download `surfer.m`, `pagerank.m` and `pagerankpow.m`
- `[U,G] = surfer('http://www.slashdot.org',20);`

  U = a cell array of n strings, the URLs of the nodes.

  G = an n-by-n sparse matrix with $G(i,j) = 1$ if node j is linked to node i.
- Next: `spy(G)`. This shows the nonzero structure of the connectivity matrix.
- Finally: `pagerank(U,G)` and the pagerank will be computed.

Numerics?

- what does this have to do with numerical methods?
- large sparse matrices (although Google avoids this)
- solution to a linear system (eigenvalue problem)
- Markov chains...

The Markov process is important. Let's step back and look at some statistics

- randomness
- Monte Carlo
- Markov chains
- Metropolis algorithm

Brownian Motion is an example of a Markov process:
`https://labs.minutelabs.io/Brownian-Motion/` (more later)

Why

- Central to the PageRank (and many many other applications in finance, science, informatics, etc) is that we randomly process something
- what we want to know is “on average” what is likely to happen
- what would happen if we have an infinite number of samples?
- next: eigenvalues (Power Method), SVD, then Monte Carlo

Goal

- Find $x = Ax$ and the elements of x are Google's PageRank.
- For a matrix A, the scalar-vector pairs (λ, v) such that $Av = \lambda v$ are eigenvalue-eigenvectors.
- Connection to Lecture 25: $v^{(n)} = P^n v^{(0)}$ repeatedly multiplies by a matrix. For a transition matrix (columns sum to 1) the dominant eigenvalue is $\lambda_1 = 1$, and the steady state is its eigenvector.
- Topic #1: Power Method
- Topic #2: Singular Value Decomposition (SVD)

Power Method

Suppose that A is $n \times n$ and that the eigenvalues are ordered:

$$|\lambda_1| > |\lambda_2| \geqslant |\lambda_3| \geqslant \cdots \geqslant |\lambda_n|$$

Assuming A is diagonalizable, we have a linearly independent set of eigenvectors v_i such that $Av_i = \lambda_i v_i$.

Goal

Computing the value of the largest (in magnitude) eigenvalue, λ_1.

Power Method

Take a guess at the associated eigenvector, $x^{(0)}$. We know

$$x^{(0)} = c_1 v_1 + \cdots + c_n v_n$$

Since the guess was random, start with all $c_j = 1$ (we only need $c_1 \neq 0$):

$$x^{(0)} = v_1 + \cdots + v_n$$

Then compute

$$\begin{aligned} x^{(1)} &= Ax^{(0)} \\ x^{(2)} &= Ax^{(1)} \\ x^{(3)} &= Ax^{(2)} \\ &\vdots \\ x^{(k+1)} &= Ax^{(k)} \end{aligned}$$

Power Method

Or $x^{(k)} = A^k x^{(0)}$. Or

$$\begin{aligned} x^{(k)} &= A^k x^{(0)} \\ &= A^k v_1 + \cdots + A^k v_n \\ &= \lambda_1^k v_1 + \cdots + \lambda_n^k v_n \end{aligned}$$

And this can be written as

$$x^{(k)} = \lambda_1^k \left(v_1 + \left(\frac{\lambda_2}{\lambda_1} \right)^k v_2 + \cdots + \left(\frac{\lambda_n}{\lambda_1} \right)^k v_n \right)$$

So as $k \to \infty$, we are left with

$$x^{(k)} \approx \lambda_1^k v_1$$

The error shrinks like $|\lambda_2/\lambda_1|^k$: convergence is slow when $|\lambda_2| \approx |\lambda_1|$.

The Power Method (with normalization)

Without scaling, $\|x^{(k)}\|$ grows (or decays) like $|\lambda_1|^k$, so we normalize each step.

```
x = initial guess with ||x||_∞ = 1
for k = 1 to kmax
    y = Ax
    r = φ(y)/φ(x)
    x = y/||y||_∞
```

- ϕ is any fixed linear functional (it picks out one number from a vector); often $\phi(x) = x_1$ is sufficient, as long as $\phi(x) \neq 0$.
- $r \to \lambda_1$ and $x \to$ an eigenvector for λ_1 (scaled so that $\|x\|_\infty = 1$).

Inverse Power Method

- We now want to find the smallest (in magnitude) eigenvalue
- $Av = \lambda v \quad \Rightarrow \quad A^{-1}v = \frac{1}{\lambda}v$
- So "apply" power method to A^{-1} (assuming A is nonsingular with a distinct smallest eigenvalue)
- $x^{(k+1)} = A^{-1}x^{(k)}$, and $r \to 1/\lambda_n$
- Never form A^{-1}; easier with $A = LU$ (factor once, reuse every step)
- Update RHS and solve:

$$Ly = x^{(k)}, \qquad Ux^{(k+1)} = y$$

(i.e. $Ux^{(k+1)} = L^{-1}x^{(k)}$), then normalize as before.

Coming Up

- Singular Value Decomposition (SVD)
- Least-Squares
- Monte Carlo Simulations